

01— Introduction to Forward Deployed Engineering

• What Is Forward Deployed Engineering ?

- **Forward Deployed Engineering (FDE)** is a software-engineering model where engineers work very closely with **customers**, often directly inside their workflows to build and deploy solutions to **real operational problems**.
- A **Forward Deployed Engineer (FDE)** sits somewhere between a **software engineer**, **solutions architect**, and **product engineer**.
- Instead of receiving a fully specified feature and implementing it, they might hear the issue then **investigates** the workflow, **talks with users**, accesses the **relevant systems** and **data**, figures out what **actually needs** to be built, **implements** it, **deploys** it, and iterates based on real-world usage.
- **FDE = software engineering + customer proximity + ownership of the outcome.**

• Why FDE ?

- In the **AI startup environment**, the challenge is **not training** or **exposing a model**; rather, it lies in **integrating** that model into an enterprise's **complex data**, **systems**, **permissions**, and **workflows** so it functions reliably. And that's what exactly a forward deployed engineer do.

• Origin of Ai ?

- **Before computers: the idea of artificial minds:**
 - For centuries, people have imagined artificial beings able to think.
 - Ancient myths included mechanical or synthetic figures, yet such ideas stayed philosophical or fictional rather than scientific.
 - The main question arised: **Can human reasoning be expressed as rules a machine can run?** This idea became more plausible as mathematics, logic, and computing improved.
- **1930s–1940s: foundations of computing:**
 - Alan Turing established computation foundations and proposed the **Turing Test** in his **1950** paper asking **whether machines could think**.
 - Electronic computers simultaneously made **machine computation** practical.
- **1956: AI gets its name:**
 - The event most commonly considered the **birth of AI** as a formal academic field was the **Dartmouth Summer Research Project** on Artificial Intelligence in **1956**.
 - John McCarthy coined the term "**Artificial Intelligence**".
 - Their basic idea was ambitious:
 - Machines could potentially be made to **simulate** aspects of human intelligence such as **reasoning**, **learning**, and **problem solving**.
- **Early AI: 1950s–1970s:**
 - Early researchers mostly used symbolic AI. Instead of training a machine on **huge datasets**, they explicitly **programmed knowledge** and **rules**.
 - For example:
 - IF** temperature > 38°C
 - AND** symptoms = fever
 - THEN** suspect illness

01— Introduction to Forward Deployed Engineering

- **1980s–2000s: Machine Learning becomes important:**

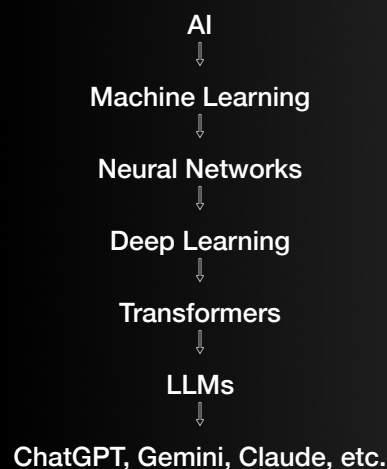
- The approach shifted from “Tell the computer all the rules” to “Give it data and let it learn patterns”. This is the foundation of **Machine Learning (ML)**.
- Traditional programming uses **predefined data** and **rules** to produce **results**, while **machine learning** uses **data** and **related outputs** to develop **predictive models**.
- Typical algorithms include **decision trees**, **neural networks**, **support vector machines**, and ensemble methods.

- **2010s: Deep Learning revolution:**

- AI systems became much better because of **massive datasets**, **powerful GPUs**, **better neural networks**, and **improved training methods**.
- Deep learning helped AI make big improvements in **image recognition**, **speech recognition**, **language translation**, **natural language processing**, and **gaming**.
- One important milestone was **AlexNet** in **2012**, which showed how powerful deep neural networks could be for recognizing images and helped to start the modern deep learning revolution.

- **2017 onward: Transformers:**

- A major turning point in modern AI came in 2017 with the research paper “**Attention Is All You Need**”.
- It introduced the **Transformer architecture**, which made it easier for **AI models** to **understand relationships** between **words** and **other types** of data.
- **Transformers** quickly became the foundation of many modern AI systems, especially **Large Language Models (LLMs)**.
- The development can be summarized as:

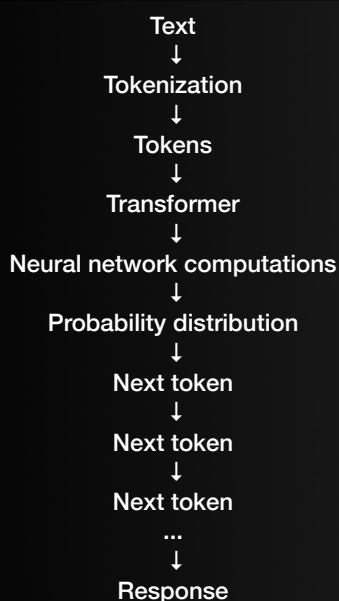


- **RNN (Recurrent Neural Network):**

- An RNN is a neural network designed to process **sequential data**, where the order of information matters.
- Ex:
 - Text → words come in a sequence
 - Speech → sounds come in a sequence
 - Stock prices → values come over time
 - Sensor data → measurements come over time
- The key idea is memory, Suppose we have: “I live in India and I speak ____.” To predict the next word, the model needs information from previous words.
- An RNN processes the sentence one word at a time: I → live → in → India → and → I → speak → ?

01— Introduction to Forward Deployed Engineering

- Internally, it maintains a hidden state which carries the summarized information from earlier words to later steps. On every word it updates the state.
- Problem with basic RNNs :**
 - Basic RNNs struggle to remember information that occurred **far earlier** in a sequence.
 - Example: "The **dog**, which I saw several years ago when I was visiting my uncle, **was** very friendly." The model needs to connect **dog** → **was**, despite many words in between.
 - This is related to the **vanishing-gradient** problem. That's why architectures like **LSTM**, **GRU** were developed.
- What is Transformer ? :**
 - It is Introduced in a research paper "**Attention Is All You Need**" published in 2017.
 - Transformer architecture is based on the mechanism of **attention**. Unlike recurrent neural networks (RNNs), which process **sequences** sequentially (e.g., word1 → word2 → word3 → word4 → ...).
 - Attention** enables the model to evaluate **interdependencies** across the entire sequence.
 - For example, in the sentence "The animal didn't cross the road because **it** was tired," the model needs to figure out what "**it**" refers to. **Attention** makes it easier to notice how words connect !
- LLM (Large Language Model) :**
 - An LLM is a machine-learning model trained on enormous amounts of text to learn patterns in language. Ex: GPT models, Claude, Gemini, Llama.
 - The basic task can be surprisingly simple: **Predict the next token**.
 - For example:
 - The capital of France is _____. The model might predicts Paris.
 - I went to the store to buy some _____. Possible prediction is milk.
 - But modern LLMs learn vastly more complex patterns than simple word completion.
- How does ChatGPT fit into this ? :**



RNN

- sequential processing
- hidden state / recurrence
- older NLP architecture

Transformer

- attention
- highly parallel training
- foundation of modern LLMs

LLM

- usually Transformer-based
- enormous number of parameters
- trained on huge datasets
- capable of language generation, reasoning-like behavior, summarization, coding, translation, etc.